Неделя 2. Лонгрид. Pandas-1: введение, базовые операции и регулярные выражения

- **Уровень сложности:** Базовый
- **Время прочтения:** ~25 минут
- **Цель материала:** Освоить архитектуру ключевых объектов библиотеки Pandas (`Series` и `DataFrame`), научиться загружать данные, делать выборки с помощью `.loc` и `.iloc` , фильтровать датафреймы булевыми масками, а также очищать и парсить неструктурированные текстовые данные с помощью методов `.str` и регулярных выражений (Regex).

Контекст

В реальной практике аналитика данные поступают в неидеальном виде. Информация о пользователях, платежах и событиях хранится в текстовых файлах или базах данных, содержит пропуски (`NaN`), лишние пробелы и служебные теги (например, `ID_102501_MSK_SUCCESS`).

Стандартные списки и циклы Python (`for`) работают с миллионами таких записей крайне медленно. Библиотека **Pandas** предоставляет векторизованные структуры данных, позволяющие очищать, трансформировать и сегментировать таблицы за считанные строки кода.

Применение

Навыки этой недели являются фундаментом работы с данными:

- **Создание и выгрузка таблиц:** сборка датафреймов из словарей, сохранение и импорт из Excel/CSV.
- **Первичная гигиена (Data Health Check):** поиск пропущенных значений, проверка типов данных и переименование столбцов.
- **Сегментация и расчеты:** создание новых метрик, фильтрация по сложным бизнес-условиям.
- **Очистка и Data Mining:** извлечение телефонов, почт и кодов ошибок из текстов с помощью Regex.

Краткий план

1. Архитектура Pandas: Series и DataFrame
2. Создание структур вручную и импорт данных
3. Переименование колонок, индексы и управление памятью (`inplace`)
4. Создание новых колонок и удаление данных (`.drop()`)
5. Работа с пропущенными значениями (`NaN`)
6. Адресация и выборки: `.loc` против `.iloc`

7. Сегментация данных через сложные булевы маски
8. Ранжирование, сортировка и переиндексация
9. Векторная работа со строками через `.str`
10. Парсинг текста с помощью регулярных выражений (Regex)
11. Если кратко
12. Проверь себя

---

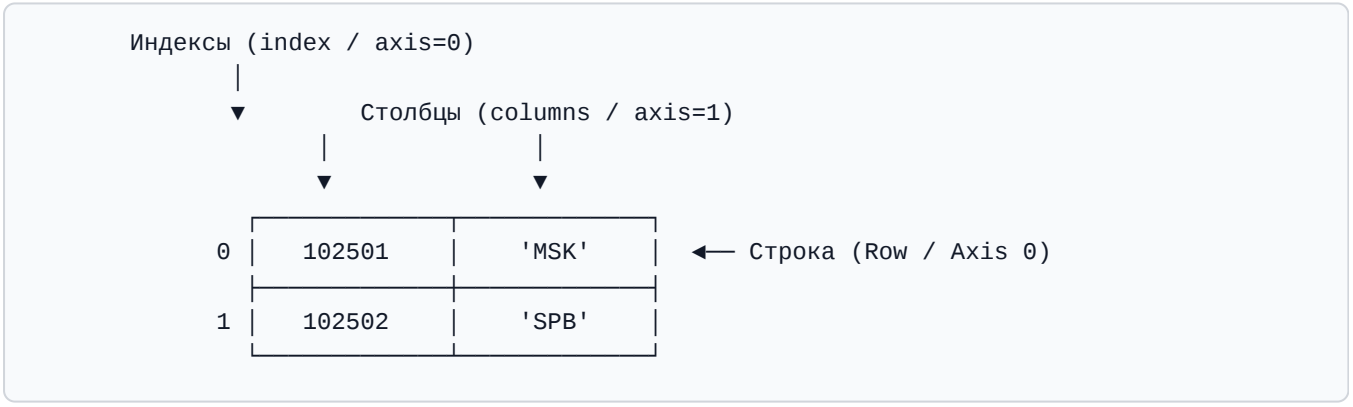
1. Архитектура Pandas: Series и DataFrame

В основе библиотеки Pandas лежат два ключевых объекта: **Series** и **DataFrame**.

Концепт

- **Series (Серия)** — одномерный индексированный массив, способный хранить данные любого единого типа (числа, строки, даты, булевы значения).
- **DataFrame (Датафрейм)** — двумерная структура данных (таблица), состоящая из упорядоченного набора столбцов, каждый из которых представляет собой объект `Series`.

Принцип связи объектов: $$\text{DataFrame} = \begin{bmatrix} \text{Series}_1 & \text{Series}_2 & \dots & \text{Series}_n \end{bmatrix}$$



> **Важно:** Все элементы внутри одной `Series` должны иметь один и тот же тип данных (`dtype`). В `DataFrame` разные столбцы могут иметь разные типы, но внутри одного столбца тип данных строго гомогенен.

Проверь себя: Можно ли в одной колонке (объекте `Series`) сохранить одновременно числа, строки и булевы значения без потери производительности? 👉 Перейти к ответу

---

2. Создание структур вручную и импорт данных

На практике данные можно создавать «с нуля» из стандартных структур Python или импортировать из внешних файлов.

Ручное создание Pandas-объектов

```
import pandas as pd

# 1. Создание Series из списка
user_ids = pd.Series([101, 102, 103], name='user_id')

# 2. Создание DataFrame из словаря (Dictionary) — самый частый способ
data = {
    'user_id': [101, 102, 103],
    'city': ['Москва', 'Санкт-Петербург', 'Казань'],
    'amount': [5000, 1200, 3400]
}
df_manual = pd.DataFrame(data)
```

Импорт из файлов и первичный аудит (Data Health Check)

Вы выступаете в роли продуктового аналитика финтех-платформы. Вы загружаете выгрузку клиентских транзакций `payments.xlsx`:

```
# Загрузка Excel-файла
df_payments = pd.read_excel('payments.xlsx')

# Первичный осмотр датафрейма
print(f"Размерность таблицы (строк, колонок): {df_payments.shape}")
df_payments.head(3)
```

Ключевые методы первичного аудита:

- `df.shape` — возвращает кортеж с количеством строк и столбцов $(N_{\text{rows}}, N_{\text{cols}})$.
- `df.info()` — выводит информацию о типах данных, количестве непустых (`non-null`) значений и памяти.
- `df.describe()` — рассчитывает основные статистические показатели для числовых колонок (среднее, медиану, стандартное отклонение, минимум, максимум, квантили).

Проверь себя: Вы загрузили датасет. Вызов `df.shape` вернул `(5000, 12)`. Вызов `df.info()` показал, что в колонке `email` 4500 non-null значений. Сколько пропусков в этой колонке? 👉

Перейти к ответу

---

3. Переименование колонок, индексы и управление памятью (`inplace`)

Переименование столбцов

При загрузке из внешних систем названия колонок часто содержат некириллические символы или пробелы. Для удобства их приводят к единому стандарту `snake_case`.

```
# Переименование отдельных колонок через словарь
df_payments = df_payments.rename(columns={
    'PAYMENT ID': 'payment_id',
    'User Region': 'region'
})
```

Установка и сброс индексов

Индексом может быть не только порядковый номер (0, 1, 2...), но и уникальный идентификатор (например, `user_id` или `payment_id`).

```
# Установка колонки в качестве индекса
df_indexed = df_payments.set_index('payment_id')

# Возврат индекса обратно в колонки
df_reset = df_indexed.reset_index()
```

Важное концептуальное правило: `inplace=True` vs Присвоение

Большинство методов Pandas (например, `.rename()`, `.sort_values()`, `.dropna()`) **не меняют исходный датафрейм на месте**, а возвращают его измененную копию.

Чтобы сохранить изменения, используйте один из двух подходов:

```
# Вариант 1 (Рекомендуемый): Переприсвоение переменной
df_payments = df_payments.rename(columns={'region': 'city_region'})

# Вариант 2: Использование параметра inplace=True
df_payments.rename(columns={'region': 'city_region'}, inplace=True)
```

Проверь себя: Вы написали код `df.rename(columns={'A': 'B'})`, но при следующем вызове `df.head()` колонка `A` осталась на месте. Назовите два способа это исправить. 👉 Перейти к ответу

---

4. Создание новых колонок и удаление данных (`.drop()`)

Добавление новых расчетных столбцов

В векторизованных вычислениях математические операции можно проводить напрямую над колонками:

```
# Расчет комиссии (1.5% от суммы платежа)
df_payments['fee'] = df_payments['amount'] * 0.015

# Расчет итоговой суммы с учетом комиссии
df_payments['total_amount'] = df_payments['amount'] + df_payments['fee']
```

Удаление колонок и строк

Для удаления ненужных признаков используется метод `.drop()`. Указывать ось обязательно: `axis=1` для колонок и `axis=0` для строк.

```
# Удаление столбцов
df_payments = df_payments.drop(columns=['fee', 'total_amount'])

# Удаление строк по их индексам (например, строк 0 и 1)
df_payments = df_payments.drop(index=[0, 1])
```

Проверь себя: Вы хотите удалить колонку `tmp_calc`. Напишите метод. Что произойдет, если использовать синтаксис `df.drop('tmp_calc')` и забыть указать ось? 👉 Перейти к ответу

---

5. Работа с пропущенными значениями (`NaN`)

В реальных датасетах часто встречаются пропуски (`NaN` — Not a Number). Pandas предоставляет набор векторизованных методов для работы с ними.

Поиск пропусков

```
# Проверка наличия пропусков во всем датафрейме (возвращает количество NaN в каждой колонке)
print(df_payments.isna().sum())
```

Обработка и заполнение пропусков

В зависимости от бизнес-логики пропуски либо удаляют, либо заполняют нейтральными/расчетными значениями.

```
# 1. Заполнение пропусков константой или медианой
df_payments['city'] = df_payments['city'].fillna('Не указано')

# 2. Удаление строк, содержащих NaN хотя бы в одной из колонок
df_clean = df_payments.dropna()

# 3. Удаление строк, где NaN находится в конкретной важной колонке
df_clean_amount = df_payments.dropna(subset=['amount'])
```

Проверь себя: У вас есть колонка `amount` (суммы покупок) с пропусками у тех, кто ничего не купил. В чем будет бизнес-разница между применением `df['amount'].fillna(0)` и `df.dropna(subset=['amount'])` перед расчетом среднего чека? 👉 Перейти к ответу

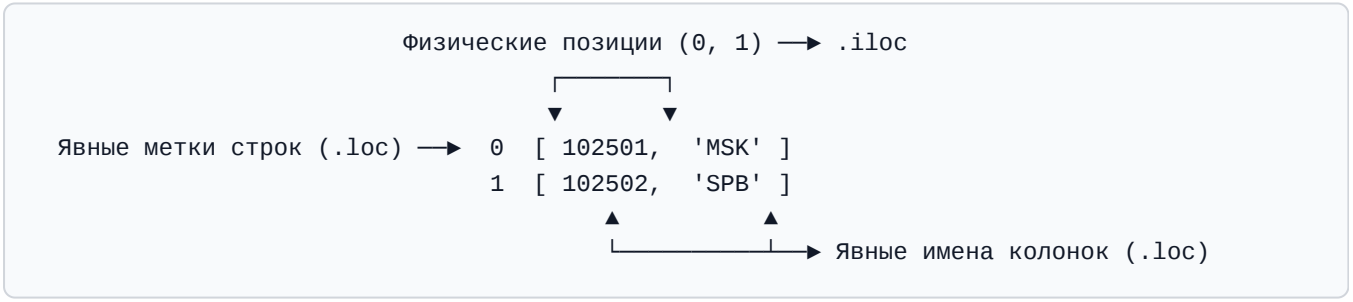
---

6. Адресация и выборки: `.loc` против `.iloc`

Одна из самых распространенных ошибок — путаница между позиционной и индексной адресацией при срезах данных.

Разбор механизмов

1. **`.iloc[]` (Integer Location)** — выборка исключительно по **физическим целым позициям** (индексам строк и колонок), начиная с `0`.
2. **`.loc[]` (Label Location)** — выборка по **явным меткам** (названиям строк и колонок).



Сравнительный разбор вызова:

```
# Извлечение первой строки и первых двух столбцов ПО ПОЗИЦИИ
df_payments.iloc[0:1, 0:2]

# Извлечение строк по явным индексам и именам колонок ПО МЕТКАМ
df_payments.loc[0:1, ['payment_id', 'amount']]
```

> **Важно:** Помните ключевое отличие в границах срезов:
>
> - В `.iloc[0:5]` верхняя граница **не включается** (возьмутся элементы 0, 1, 2, 3, 4).

> - В `.loc[0:5]` верхняя граница **включается полностью** (возьмутся элементы с метками 0, 1, 2, 3, 4, 5).

Проверь себя: Есть датафрейм из 100 строк. Чем срез `df.iloc[0:5]` концептуально отличается от `df.loc[0:5]` ? 👉 Перейти к ответу

---

7. Сегментация данных через сложные булевы маски

Фильтрация данных в Pandas базируется на **булевых масках** — векторизованных массивах из `True` и `False`.

Построение составных условий

В Pandas операции выполняются векторно над столбцами, поэтому вместо ключевых слов `and` / `or` используются **побитовые операторы**:

- `&` — Побитовое И (**AND**)
- `|` — Побитовое ИЛИ (**OR**)
- `~` — Побитовое НЕ (**NOT / Отрицание**)

$$\text{Mask} = (\text{Series}_1 > \text{val}_1) \ \& \ (\text{Series}_2 == \text{val}_2)$$

> **Важно:** Каждое отдельное условие внутри булевой маски **обязательно оборачивается в круглые скобки `()`**. В противном случае из-за приоритета операторов Python вызовет ошибку `TypeError`.

```
# Фильтрация клиентов с чеком > 5000 рублей, оплативших с ПК ('pc')
mask_vip = (df_payments['amount'] > 5000) & (df_payments['device'] == 'pc')
df_vip = df_payments[mask_vip]
```

Проверь себя: Аналитик написал фильтр: `df[df['age'] > 18 & df['status'] == 'VIP']`. Код выдает ошибку `TypeError`. Почему? 👉 Перейти к ответу

---

8. Ранжирование, сортировка и переиндексация

После фильтрации сегмента данные необходимо упорядочить и переиндексировать.

```
# Сортировка по региону (по алфавиту) и по убыванию чека
df_sorted = df_payments.sort_values(
    by=['region', 'amount'],
    ascending=[True, False]
)

# Получение ТОП-5 чеков с полным сбросом старых индексов
df_top5 = df_sorted.head(5).reset_index(drop=True)
```

Проверь себя: Вы отсортировали таблицу по убыванию чека и вывели топ-3: `df_top = df.sort_values('amount', ascending=False).head(3)`. Индексы этих строк теперь выглядят так: `[42, 10, 99]`. Каким методом привести индексы к порядку `[0, 1, 2]` без создания лишних колонок? 👉 Перейти к ответу

---

9. Векторная работа со строками через `.str`

В реальных данных текстовые поля часто содержат опечатки, случайные пробелы или разный регистр. Стандартные методы Python нельзя применить к Series напрямую, поэтому Pandas использует аксессуар `.str`.

Ключевые строковые методы

Метод Pandas	Что делает	Пример использования
`.str.lower()` / `.str.upper()`	Приводит текст к нижнему/верхнему регистру	`df['city'].str.lower()`
`.str.strip()`	Удаляет лишние пробелы в начале и конце строки	`df['comment'].str.strip()`
`.str.replace()`	Заменяет подстроку или шаблон на новое значение	`df['phone'].str.replace('-', '')`
`.str.split()`	Разбивает строку на список подстрок по разделителю	`df['email'].str.split('@')`
`.str.contains()`	Проверяет, содержится ли подстрока/паттерн в тексте (возвращает булеву маску)	`df['comment'].str.contains('success')`

```
# Приведение городов к единому стандарту и удаление пробелов
df_payments['city_clean'] = df_payments['city'].str.strip().str.upper()

# Быстрый отбор всех комментариев, содержащих слово 'PAYMENT'
mask_pay = df_payments['comment'].str.contains('PAYMENT', case=False, na=False)
df_pay_logs = df_payments[mask_pay]
```

Проверь себя: В колонке `city` данные заведены неряшливо: есть значения `' msk '`, `'MSK'`, `'Msk'`. Какая цепочка векторных строковых методов за одну строку кода приведет их к идеальному стандарту `'MSK'`? 👉 Перейти к ответу

---

10. Парсинг текста с помощью регулярных выражений (Regex)

Когда простой поиск по подстроке не помогает, на помощь приходят **регулярные выражения (Regex)** — язык шаблонов для поиска, извлечения и замены сложных текстовых паттернов.

Шпаргалка по спецсимволам Regex

Символ	Значение	Пример шаблона	Что найдет
`\d`	Любая цифра от 0 до 9	`\d+`	Одна или несколько цифр подряд (`102501`)
`\w`	Буква, цифра или знак подчеркивания	`\w+`	Любое слово (`MSK_2024`)
`[A-Z]`	Заглавная латинская буква	`[A-Z]{3}`	Трехбуквенный код региона (`MSK`, `SPB`)
`()`	**Группа захвата (Capture Group)**	`(\d{4})`	Выделит ровно 4 цифры в отдельную колонку
`^` / `$`	Начало / конец строки	`^ID_\d+$`	Строку, строго начинающуюся с ID_ и состоящую из цифр

Проверь себя: В столбце `log` есть записи вида `id_445_error`. Вам нужно с помощью `.str.extract()` вытащить только само число `445` в новую колонку. Куда нужно поставить круглые скобки (группу захвата) в паттерне `r'id_\d+_error'`? 👉 Перейти к ответу

Практическое применение Regex в Pandas

1. Проверка соответствия шаблону через `.str.contains()`

Проверим, соответствует ли технический комментарий шаблону `ERR_` с последующим трехзначным кодом ошибки:

```
# Фильтрация логов с ошибками вида ERR_404, ERR_500
mask_errors = df_payments['comment'].str.contains(r'ERR_\d{3}', na=False)
df_errors = df_payments[mask_errors]
```

2. Извлечение данных в новые колонки через `.str.extract()`

Представьте, что в поле `raw_log` записана строка вида: `CLIENT_12685_CITY_MSK_AMT_6903`. Нам нужно спарсить ID клиента и код города в отдельные колонки:

```
# Использование групп захвата () для извлечения данных
extracted_data = df_payments['raw_log'].str.extract(r'CLIENT_(\d+)_CITY_([A-Z]{3})')

# Присвоение результатов новым колонкам датафрейма
df_payments[['client_id_parsed', 'city_code']] = extracted_data
df_payments[['client_id_parsed', 'city_code']].head()
```

3. Замена и очистка сложного текста через `.str.replace()`

Очистим телефонные номера от любых нечисловых символов (скобок, тире, пробелов, плюсов):

```
# Удаление всего, что НЕ является цифрой (\D или [^\d])
df_payments['phone_clean'] = df_payments['phone'].str.replace(r'\D', '', regex=True)
```

Если кратко

- **Series и DataFrame** — базовые кирпичики Pandas: `Series` — одномерная колонка, `DataFrame` — двумерная таблица из колонок.
- **Импорт и ручная сборка:** Таблицы можно собирать из словарей `pd.DataFrame(dict)` или импортировать через `read_excel()` / `read_csv()`.
- **Переприсвоение результатов:** Методы Pandas не меняют датафрейм «на месте» без переприсвоения (`df = df.method()`) или передачи `inplace=True`.
- **Пропущенные значения (`NaN`)** проверяются через `.isna()` и обрабатываются через `.fillna()` или `.dropna()`.
- **Адресация:** `.iloc[]` выбирает строго по целым физическим номерам позиций, а `.loc[]` — по явным именам колонок и меткам строк.
- **Составные булевы маски** объединяются операторами `&` (И), `|` (ИЛИ), `~` (НЕ). Каждое условие **обязательно заворачивается в круглые скобки `()`**.
- **Модуль `.str` и Regex** позволяют векторизованно парсить, фильтровать и чистить неструктурированный текст без медленных циклов `for`.

Проверь себя

1. **Задание на понимание работы с памятью (`inplace`):** Студент выполнил код `df.dropna()` и `df.rename(columns={'a': 'b'})`, однако в следующих ячейках таблицы пропуски остались, а

названия колонок не изменились. *Объясните, почему так произошло и напишите правильный вариант.*

2. **Задание на понимание срезов:** Есть датафрейм `df` из 100 строк (индексы от 0 до 99). Какое количество строк вернет вызов `df.iloc[0:10]` и вызов `df.loc[0:10]`? *Поясните разницу в логике работы верхних границ.*
3. **Поиск и исправление ошибки в коде:** Студент попытался отфильтровать платежи и применить регулярное выражение, но получил ошибку `TypeError`:

```
# Ошибка: пропущены скобки и некорректно вызван Regex
df_filtered = df[df['amount'] >= 1000 & df['comment'].str.contains('ERR_\d+')]
```

Какие 2 ошибки допущены в этом коде и как его исправить? 4. **Практический мини-кейс на Regex:** В колонке `email` хранятся электронные адреса клиентов (например, `'user123@gmail.com'`). Напишите строку кода с использованием `.str.extract()`, которая извлечет домен почты (все, что идет ПОСЛЕ символа `@`) в новую колонку `domain`.

Ответы для самопроверки

Ответ на вопрос 1: Нет. `Series` требует однородных данных. Если попытаться смешать разные типы, Pandas приведет всю колонку к общему типу `object` (по сути, к строкам). Это отключит все оптимизированные векторные вычисления NumPy под капотом, и любые математические операции будут выполняться медленно, как в обычных циклах Python. Вернуться назад

Ответ на вопрос 2: 500 пропусков (`NaN`). Общее число строк — 5000, из них только 4500 содержат заполненные данные. Вернуться назад

Ответ на вопрос 3: Метод вернул изменённую *копию*, а исходный датафрейм не тронул.

1. Использовать переприсвоение: `df = df.rename(...)`.
2. Использовать специальный аргумент: `df.rename(..., inplace=True)`. Вернуться назад

Ответ на вопрос 4: Правильный код: `df = df.drop(columns=['tmp_calc'])` или `df = df.drop('tmp_calc', axis=1)`. Если забыть указать `axis=1`, Pandas по умолчанию будет искать строку (`axis=0`) с индексом `tmp_calc` и выдаст ошибку `KeyError`. Вернуться назад

Ответ на вопрос 5:

- `fillna(0)` заменит пропуски на нули. При расчете среднего значения (mean) общая сумма разделится на всех пользователей, включая "нулевых", что резко **занизит средний чек**.
- `dropna()` просто удалит неплатящих из расчетов. Вы получите корректный средний чек исключительно по платящей аудитории. Вернуться назад

Ответ на вопрос 6:

- `.iloc[0:5]` ищет по физическим позициям и **не включает** правую границу. Он вернет ровно 5 строк (позиции 0, 1, 2, 3, 4).
- `.loc[0:5]` ищет по явным меткам индекса и **включает** правую границу. Если у вас индексы названы от 0 до 99, он вернет 6 строк (метки 0, 1, 2, 3, 4, 5). Вернуться назад

Ответ на вопрос 7: В Pandas побитовые операторы (`&` , `|`) имеют более высокий приоритет выполнения, чем операторы сравнения (`>` , `==`). Python пытается сначала выполнить нелогичное действие `18 & df['status']` . Необходимо обязательно оборачивать каждое отдельное условие в круглые скобки: `df[(df['age'] > 18) & (df['status'] == 'VIP')]` . Вернуться назад

Ответ на вопрос 8: Нужно применить метод `.reset_index(drop=True)` . Параметр `drop=True` удалит старые "рваные" индексы, не позволив им превратиться в новую колонку датафрейма. Вернуться назад

Ответ на вопрос 9: Сначала нужно удалить лишние пробелы, а затем привести всё к верхнему регистру: `df['city'] = df['city'].str.strip().str.upper()` Вернуться назад

Ответ на вопрос 10: Круглые скобки нужно поставить строго вокруг того фрагмента, который мы хотим извлечь (вокруг `\d+`). Правильный код: `df['id'] = df['log'].str.extract(r'id_(\d+)_error')` Вернуться назад